
ASSIGNMENT

PROGRAMMING USING C

WEEK – 12

NAME : K.Venkateshwari
ROLL NO : 2116240801373
DEPT & SEC :ECE F

Week -12(a)

A binary number is a combination of 1s and 0s. Its n^{th} least significant digit is the n^{th} digit starting from the right starting with 1. Given a decimal number, convert it to binary and determine the value of the 4^{th} least significant digit.

```
/*
 * Complete the 'fourthBit' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER number as parameter.
 */

int fourthBit(int number)
{
    int binary[32];
    int i=0;
    while(number>0)
    {
        binary[i]=number%2;
        number/=2;
        i++;
    }
    if(i>=4)
    {
        return binary[3];
    }
    else
    return 0;
}
```

	Test	Expected	Got	
✓	printf("%d", fourthBit(32))	0	0	✓
✓	printf("%d", fourthBit(77))	1	1	✓

Passed all tests! ✓

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the p^{th} element of the list, sorted ascending. If there is no p^{th} element, return 0.

```
/*
 * Complete the 'pthFactor' function below.
 *
 * The function is expected to return a LONG_INTEGER.
 * The function accepts following parameters:
 * 1. LONG_INTEGER n
 * 2. LONG_INTEGER p
 */

long pthFactor(long n, long p)
{
    int count=0;
    for(long i=1;i<=n;++i)
    {
        if(n%i==0)
        {
            count++;
            if(count==p)
            {
                return i;
            }
        }
    }
    return 0;
}
```

	Test	Expected	Got	
✓	printf("%ld", pthFactor(10, 3))	5	5	✓
✓	printf("%ld", pthFactor(10, 5))	0	0	✓
✓	printf("%ld", pthFactor(1, 1))	1	1	✓

Passed all tests! ✓

Week – 12(b)

You are a bank account hacker. Initially you have 1 rupee in your account, and you want exactly **N** rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount **N** using these hacks.

```
/* Complete the 'myFunc' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER n as parameter.
 */
#include<stdio.h>

int myFunc(long long n)
{
    while(n>1)
    {
        if(n%20==0)
        {
            n/=20;
        }
        else if(n%10==0)
        {
            n/=10;
        }
        else
        {
            return 0;
        }
    }
    return n==1;
}

int main()
{
    int t;
    scanf("%d",&t);
    while(t-->0)
    {
        long long n;
        scanf("%lld",&n);
        printf("%d\n",myFunc(n));
    }
    return 0;
}
```

Syntax Error(s)

__tester__.c:47:5: error: redefinition of 'main'

```
47 | int main()
    |      ^~~~
```

__tester__.c:33:5: note: previous definition of 'main' with type 'int()'

```
33 | int main()
    |      ^~~~
```

Find the number of ways that a given integer, X , can be expressed as the sum of the N^{th} powers of unique, natural numbers.

For example, if $X = 13$ and $N = 2$, we have to find all combinations of unique squares adding up to 13 . The only solution is $2^2 + 3^2$.

```
/*
 * Complete the 'powerSum' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 * 1. INTEGER x
 * 2. INTEGER n
 */
#include<stdio.h>
#include<math.h>
int powerSum(int X, int N)
{
    if(X==0)
    {
        return 0;
    }
    if(X<0 || N<1)
    {
        return 0;
    }
}
```

Syntax Error(s)

```
__tester__.c: In function 'main':
__tester__.c:29:22: error: too many arguments to function 'powerSum'
 29 |         printf("%d", powerSum(10, 1, 2));
    |                        ^~~~~~
__tester__.c:15:5: note: declared here
 15 | int powerSum(int X, int N)
    |     ^~~~~~
__tester__.c: In function 'powerSum':
__tester__.c:25:1: error: control reaches end of non-void function [-Werror=return-type]
 25 | }
    | ^
cc1: all warnings being treated as errors
```